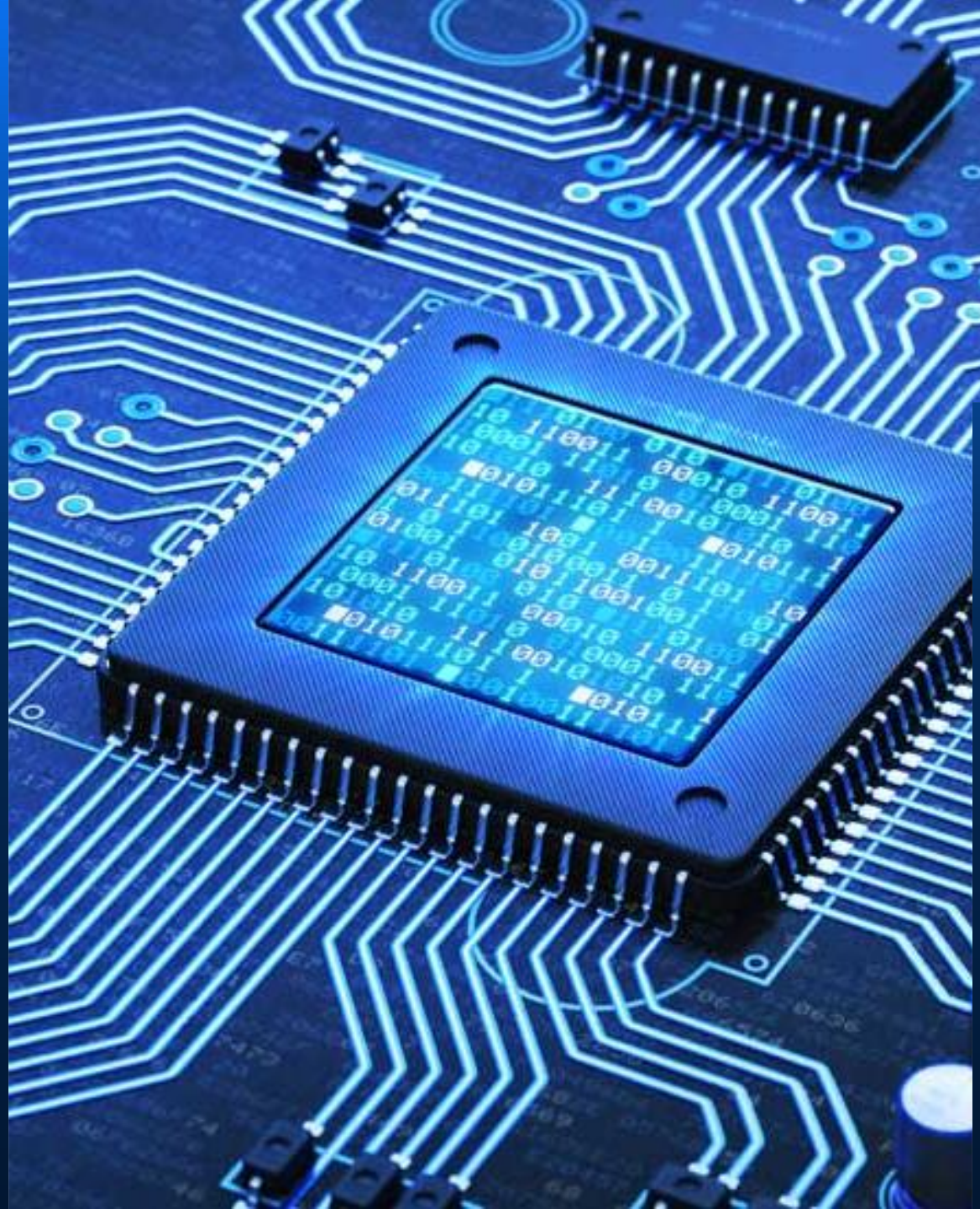


Computer Architecture

Session 7

By: Eng. Abdulkareem Mohammed

Senior HW Digital Design Engineer

This presentation is prepared from:

ARM® AMBA® 5 AHB Protocol Specification.

Agenda

AMBA AHB protocol

- # 1 Introduction
- # 2 About the protocol
- # 3 Signal Descriptions
- # 4 Transfers
- # 5 Bus Interconnection
- # 6 Slave Response Signaling

Agenda

AMBA AHB protocol

1 Introduction

2 About the protocol

3 Signal Descriptions

4 Transfers

5 Bus Interconnection

6 Slave Response Signaling

1 Introduction

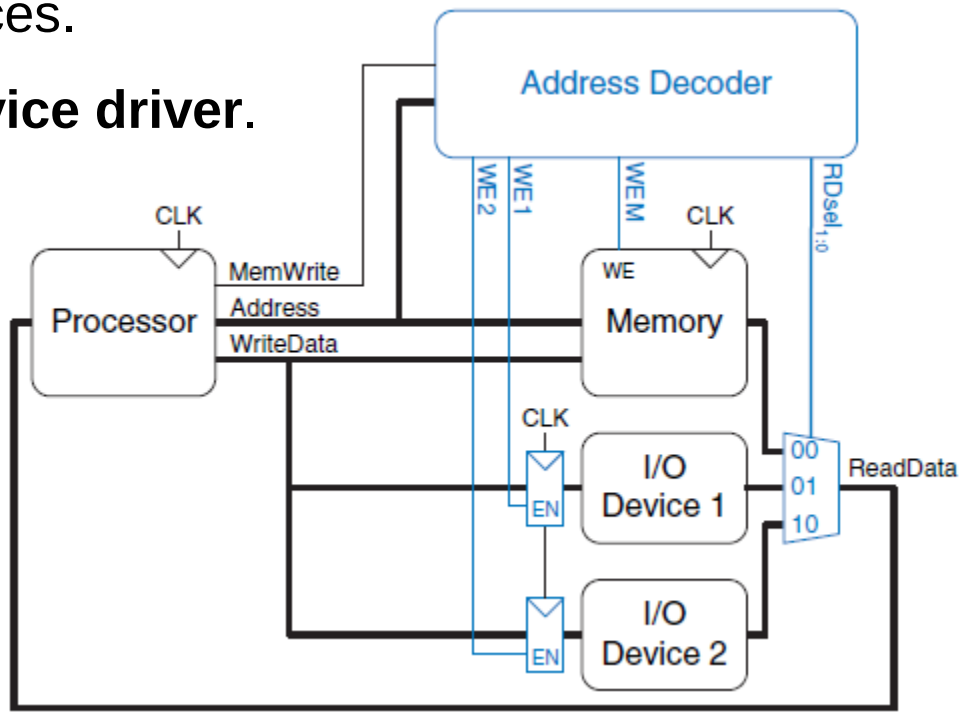
1 Introduction

- Input/Output (I/O) systems are used to connect a computer with external devices (peripherals).
- In your PC, the devices typically include keyboards, monitors, printers, and wireless networks.
- In embedded systems, devices could include a heating element, an engine's fuel injector, a solar panel positioning motors, and so forth.

- A processor accesses an I/O device using the address and data busses like accessing memory.
- A portion of the address space is dedicated to I/O devices rather than memory.
- Some addresses of the memory map are reserved for I/O devices (**memory-mapped I/O**).
- A store to the specified address sends data to the device. A load receives data from the device.

1 Introduction

- An **address decoder** determines which device communicates with the processor.
- It uses the Address and MemWrite signals to generate control signals for the rest of the hardware.
- The ReadData multiplexer selects between memory and the various I/O devices.
- Write-enabled registers hold the values written to the I/O devices.
- Software that communicates with an I/O device is called **a device driver**.
- Other programs call functions in the device driver to access the device without going into the low-level device hardware.
- The addresses associated with I/O devices are often called I/O registers.

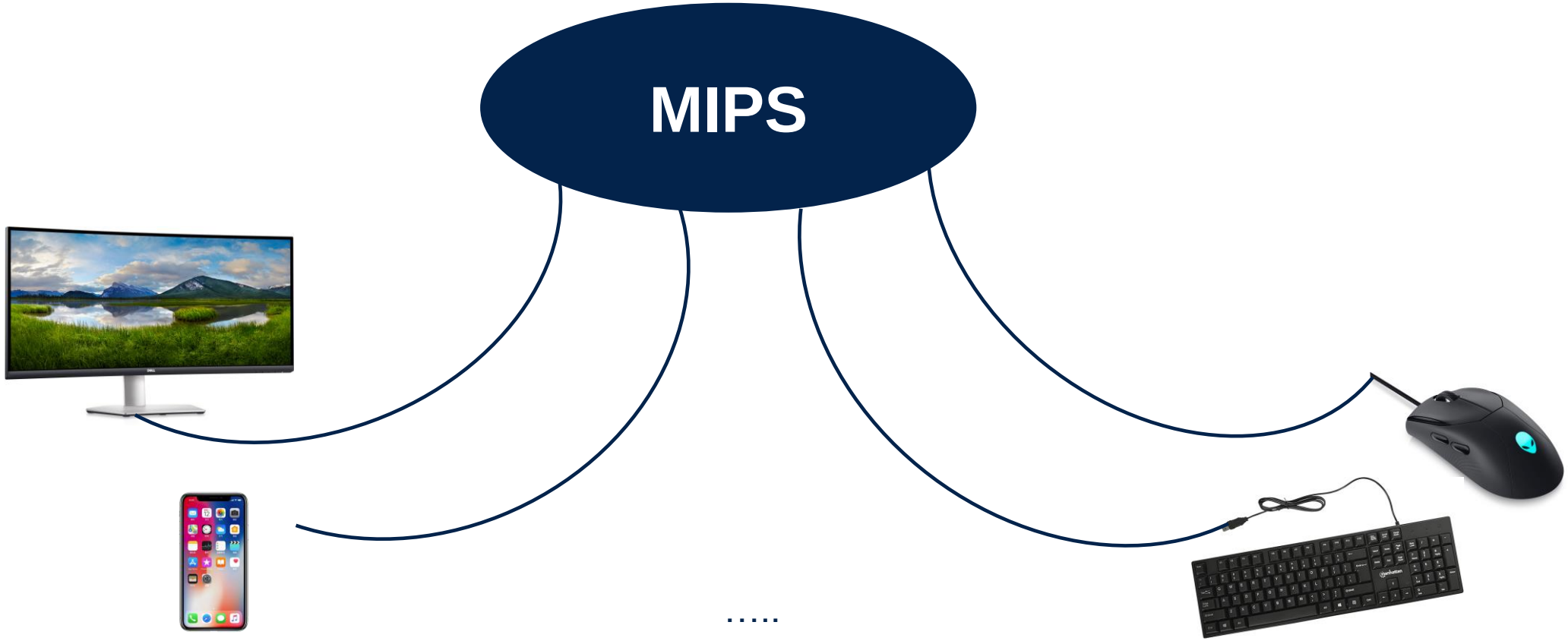


1 Introduction

- Embedded systems use a processor to control interactions with the physical environment.
- They are typically built around microcontroller units (MCUs) which combine a microprocessor with peripherals such as general-purpose digital and analog I/O pins, serial ports, timers, etc.
- Microcontrollers are generally inexpensive and are designed to minimize system cost and size by integrating most of the necessary components onto a single chip.
- Microcontrollers are classified by the size of operating data.
  8-bit microcontrollers are the smallest and least expensive.
  32-bit microcontrollers more expensive and higher performance.

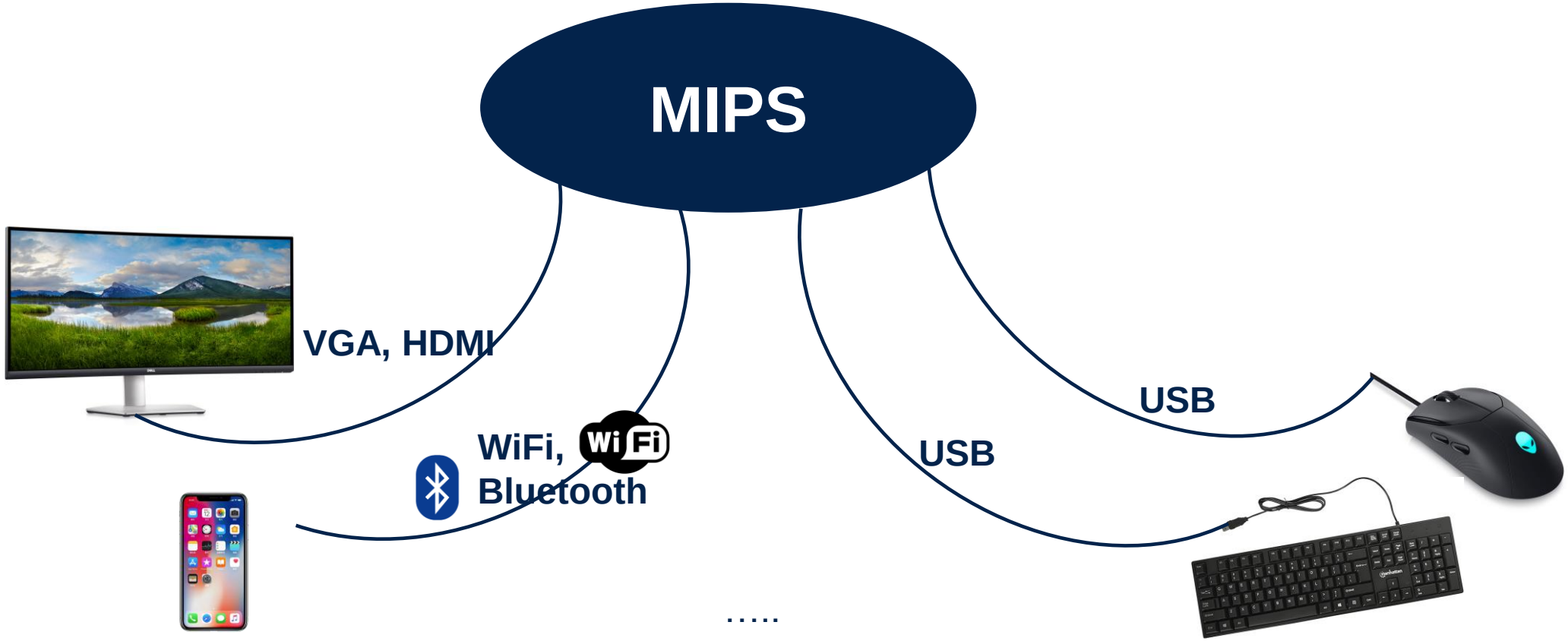
1 Introduction

- We want to connect our MIPS to the external devices!!



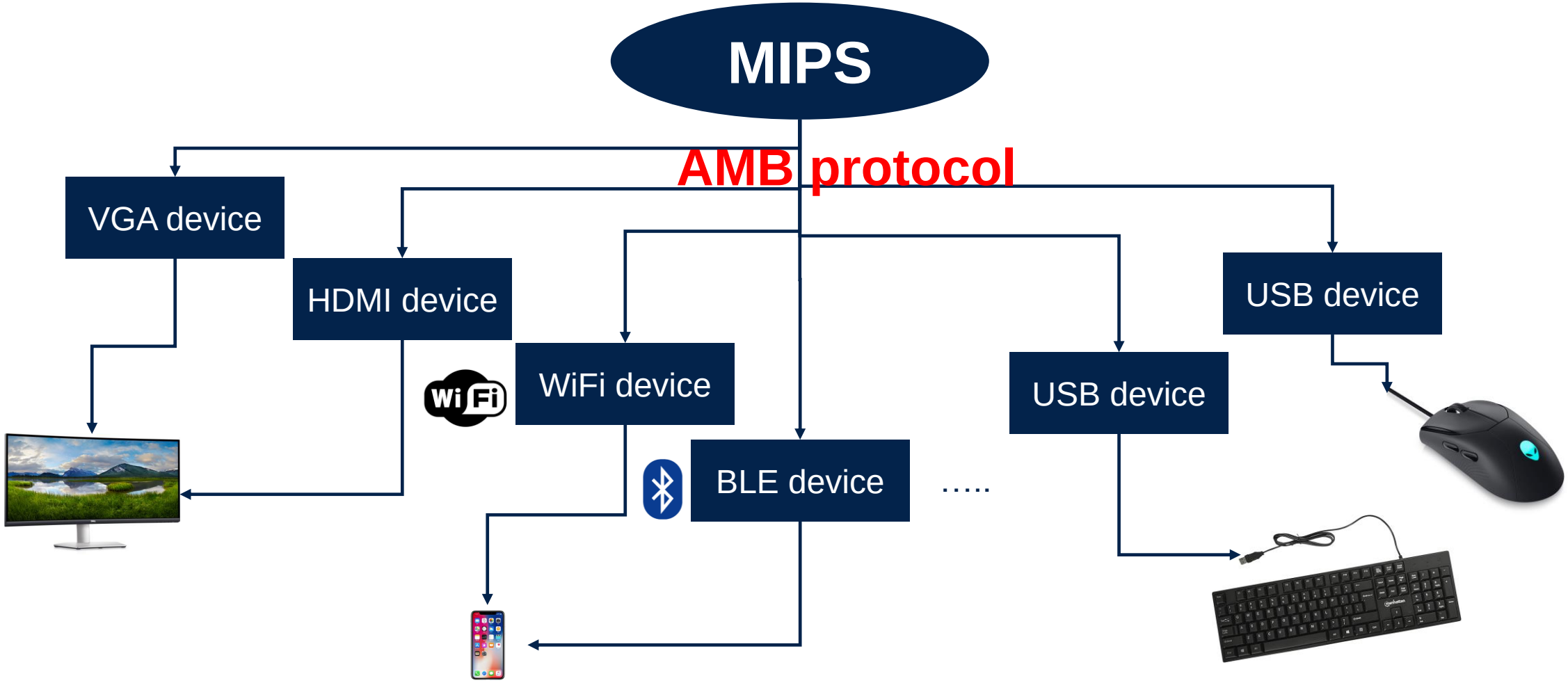
1 Introduction

- Each device has the suitable protocol which works on.

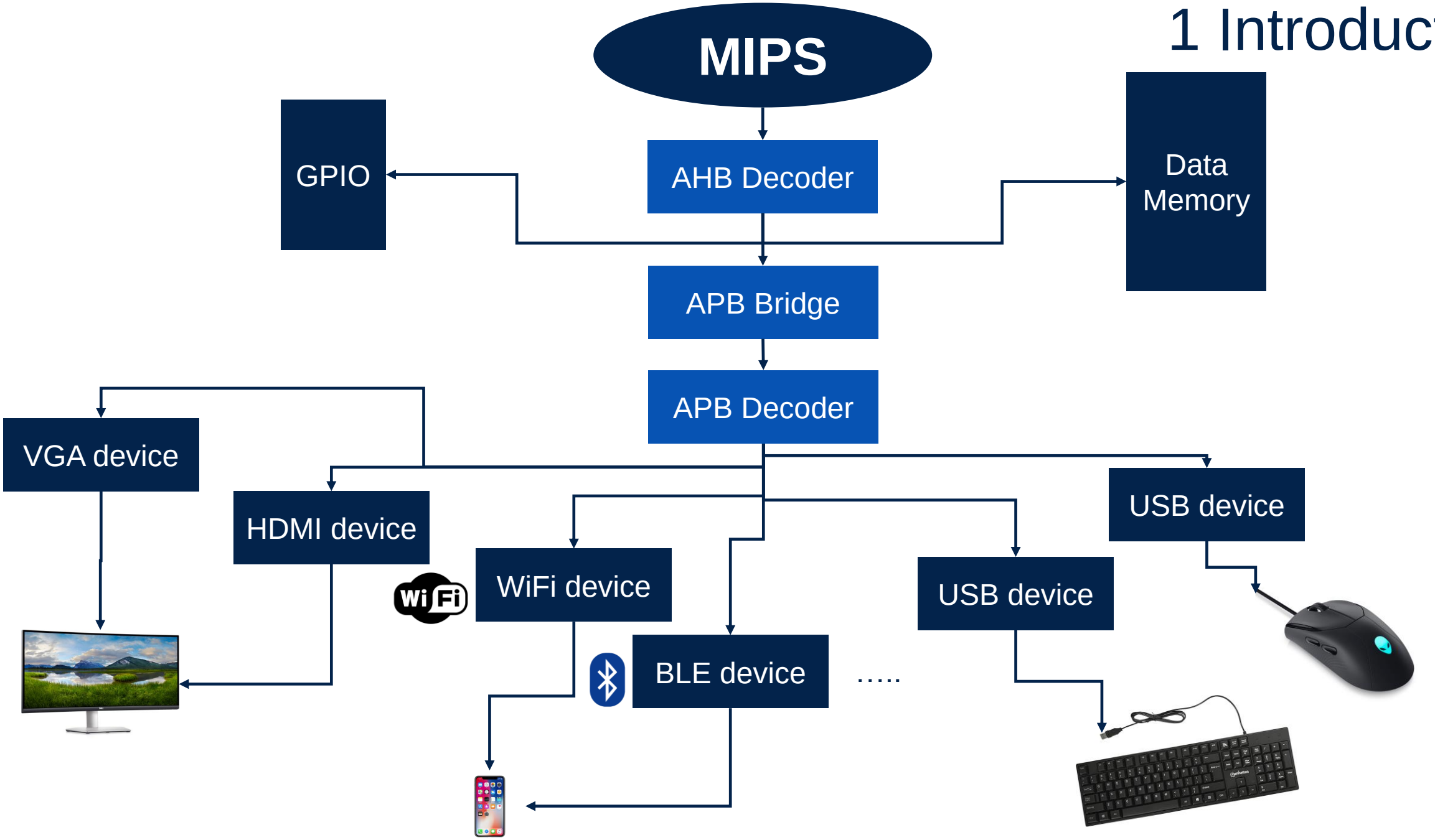


1 Introduction

- We will use a unified protocol to communicate with the different devices .



1 Introduction



Agenda

AMBA AHB protocol

1 Introduction

2 About the protocol

3 Signal Descriptions

4 Transfers

5 Bus Interconnection

6 Slave Response Signaling

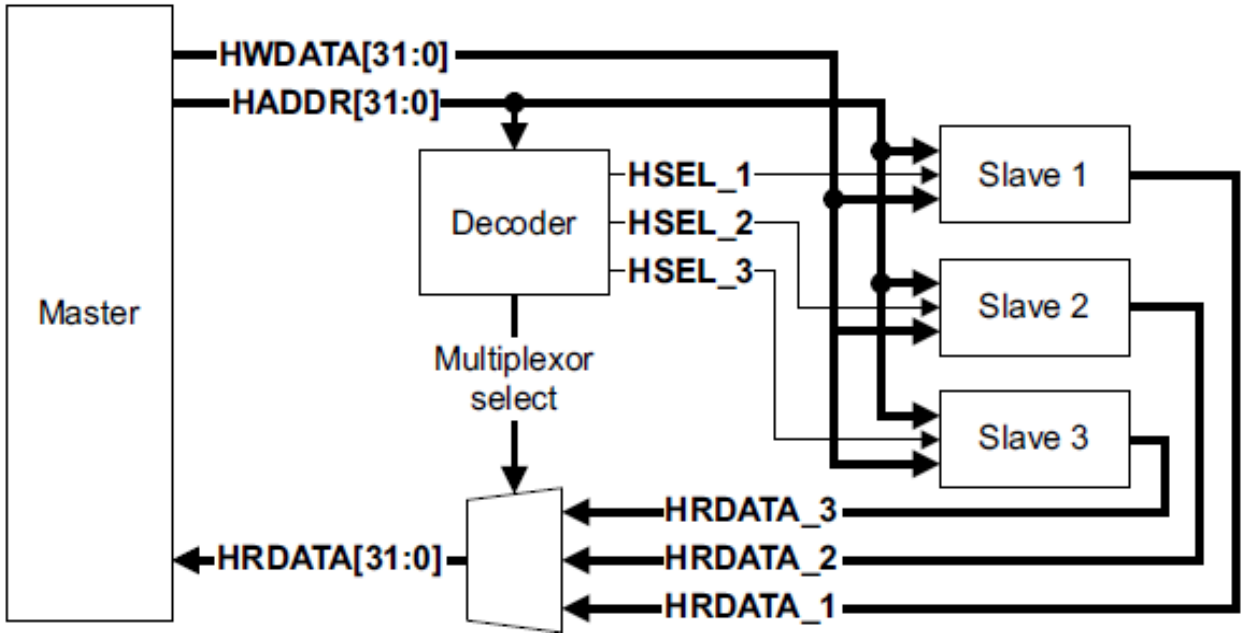
2 About the protocol

2 About the protocol

- AMBA AHB is a bus interface suitable for **high-performance** synthesizable designs.
- It defines the interface between components, such as masters, interconnects, and slaves.
- AMBA AHB implements the features required for high-performance, high clock frequency systems including:
  - Burst transfers.
  - Single clock-edge operation.
  - Non-tristate implementation.
  - Wide data bus configurations, 64, 128, 256, 512, and 1024 bits.
- The most common AHB slaves are internal memory devices, external memory interfaces, and high-bandwidth peripherals.
- Low-bandwidth peripherals typically reside on the AMBA Advanced Peripheral Bus (APB). APB bridge between the higher performance AHB and APB which is used as AHB slave.

2 About the protocol

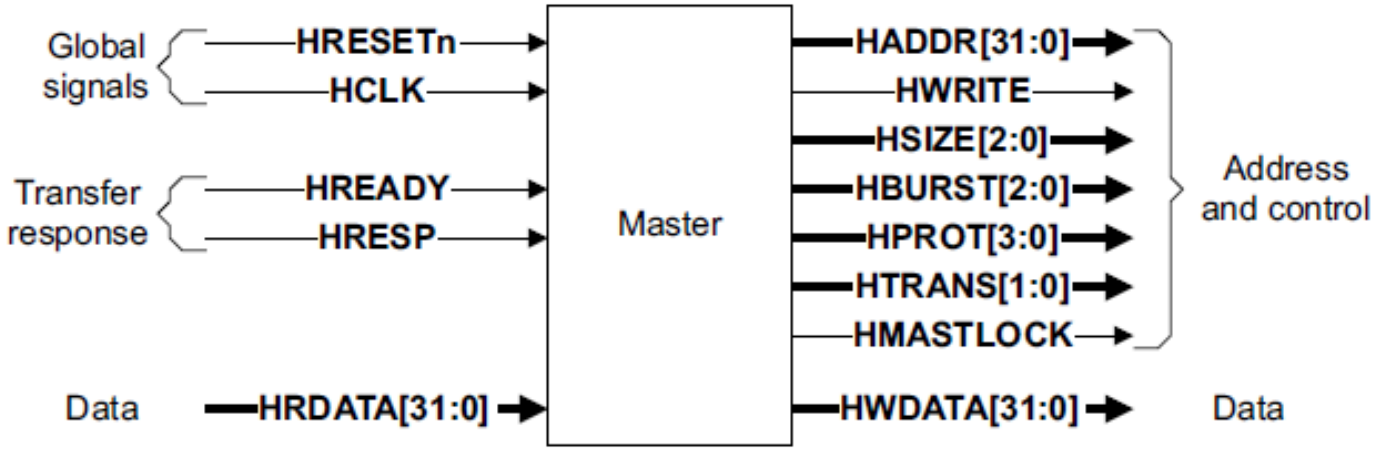
- A single master AHB system design with the AHB master and three AHB slaves.
- The bus interconnect logic consists of one **address decoder** and a **slave-to-master multiplexor**.
- The decoder monitors the address from the master so that the appropriate slave is selected.
- The multiplexor routes the corresponding slave output data back to the master.
- AHB also supports multi-master designs using an interconnect component that provides arbitration and routing signals from different masters to the appropriate slaves.



2 About the protocol

1. Master

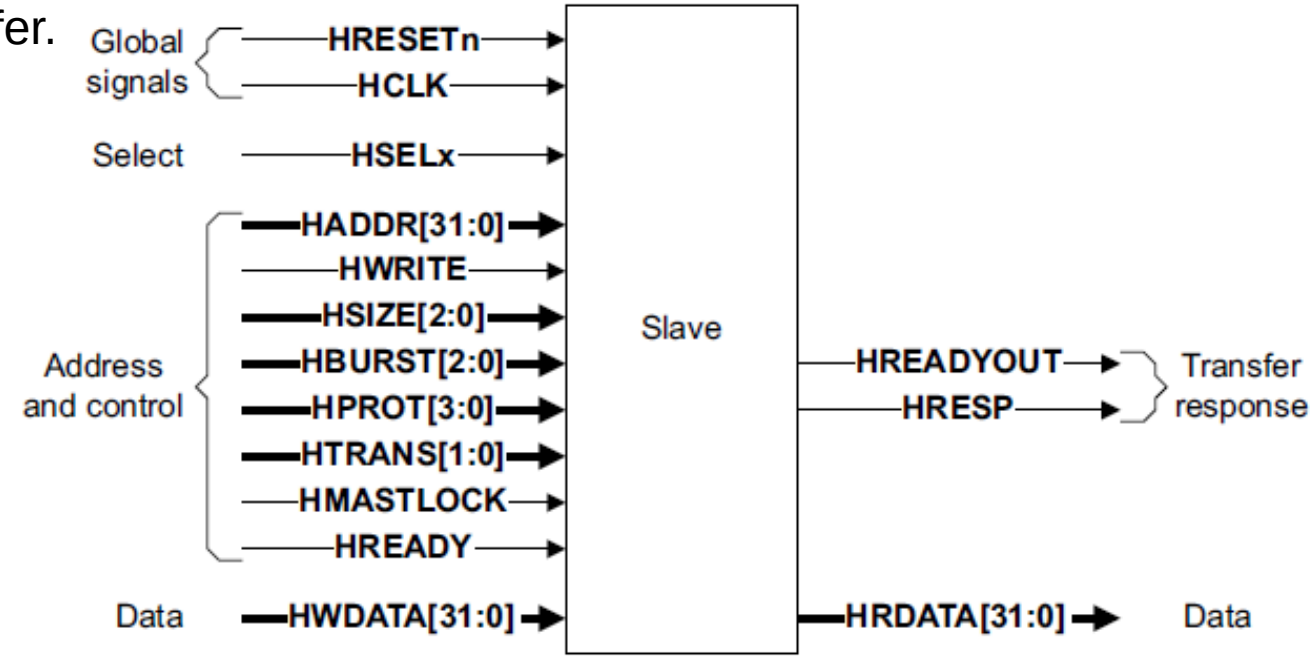
- A master provides address and control signals to initiate read and write operations.



2 About the protocol

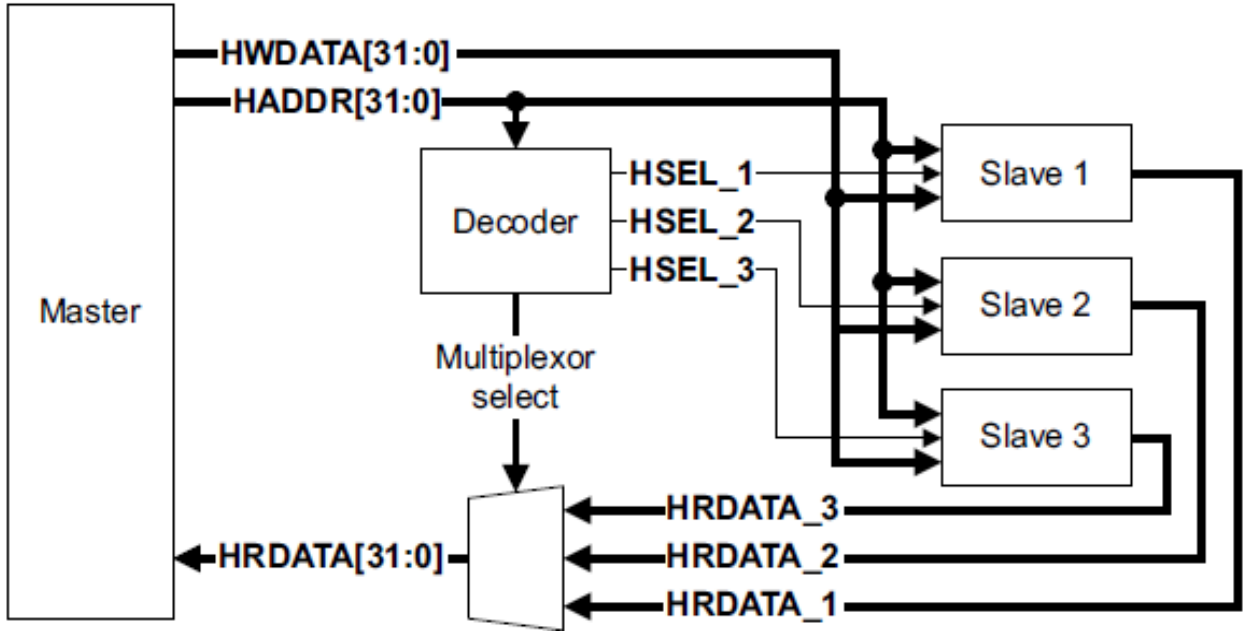
2. Slave

- A slave responds to transfers initiated by masters in the system.
- The slave uses HSELx from the decoder to control when it responds to a bus transfer.
- The slave signals back to the master:
  - The completion or extension of the bus transfer.
  - The success or failure of the bus transfer.



3. Interconnect

- An interconnect provides the connection between masters and slaves in a system.
- A single master system only requires the use of a Decoder and Multiplexor.
- A multi-master system requires the interconnect which provides arbitration and routing of address, control, and write data signals from different masters to the appropriate slaves.



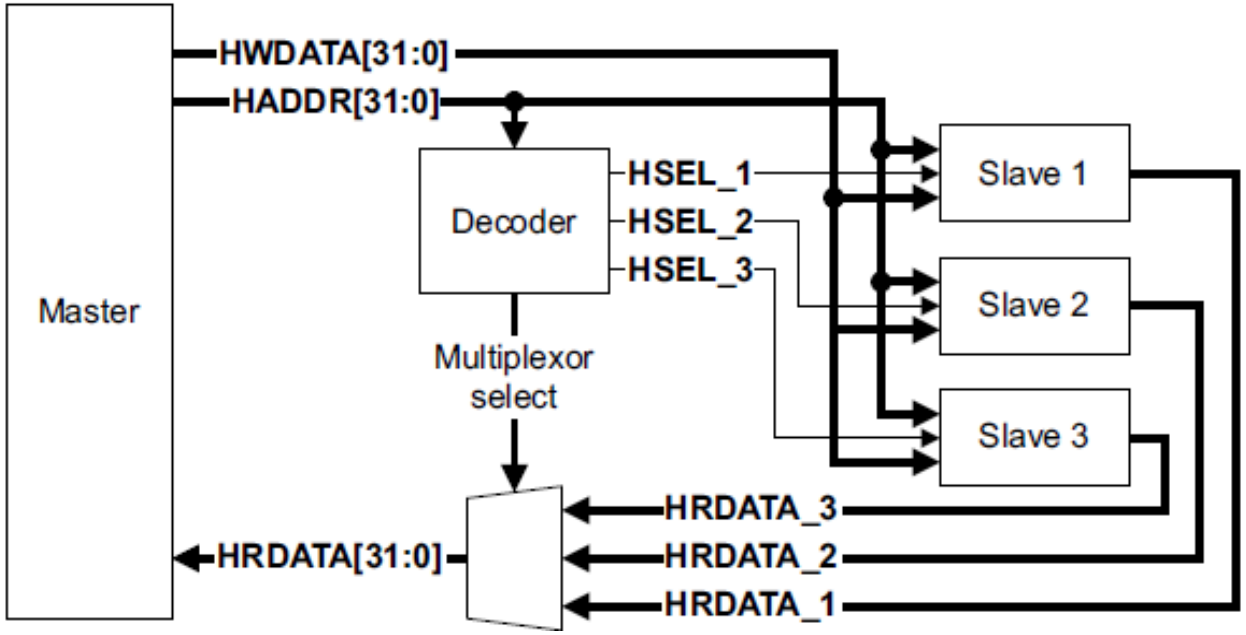
2 About the protocol

3.1 Decoder

- It decodes the address of each transfer, provides a select signal for the slave that is involved in the transfer and provides a control signal to the multiplexor.

3.2 Multiplexor

- A slave-to-master multiplexor is required to multiplex the read data bus and response signals from the slaves to the master.



Agenda

AMBA AHB protocol

- # 1 Introduction
- # 2 About the protocol
- # 3 Signal Descriptions
- # 4 Transfers
- # 5 Bus Interconnection
- # 6 Slave Response Signaling

3 Signal Descriptions

3 Signal Descriptions

- **Global signals**

Name	Source	Description
HCLK	Clock source	The bus clock times all bus transfers. All signal timings are related to the rising edge of HCLK.
HRESETn	Reset controller	The bus reset signal is active LOW and resets the system and the bus. This is the only active LOW signal.

3 Signal Descriptions

1. **Master signals**

Name	Destination	Description
HADDR[31:0]	Slave and decoder	The 32-bit system address bus.
HWDATA[31:0]	Slave	The write data bus transfers data from the master to the slaves during write operations. It can be extended to enable higher bandwidth operation.
HWRITE	Slave	It indicates the transfer direction. It has the same timing as the address signals; however, it must remain constant throughout a burst transfer.
HSIZE[2:0]	Slave	It indicates the size of the transfer which is byte, halfword, or word. The protocol allows for larger transfer sizes up to a max of 1024 bits.

3 Signal Descriptions

1. Master signals

Name	Destination	Description
HTRANS[1:0]	Slave	It indicates the transfer type of the current transfer. This can be: • IDLE • BUSY • NONSEQUENTIAL • SEQUENTIAL.
HBURST[2:0]	Slave	It indicates if the transfer is a single transfer or forms part of a burst. Fixed length bursts of 4, 8, and 16 beats are supported. The burst can be incrementing or wrapping. Incrementing bursts of undefined length are also supported.

3 Signal Descriptions

1. Master signals

Name	Destination	Description
HPROT[3:0]	Slave	The protection control signals provide additional information about a bus access and indicate how an access should be handled within a system. The signals indicate if the transfer is an opcode fetch or data access, and if the transfer is a privileged mode access or a user mode access.
HPROT[6:4]	Slave	It adds extended memory types. It is supported if the AHB Extended_Memory_Types property is True.
HNONSEC	Slave and decoder	Indicates that the current transfer is either a Non-secure transfer or a Secure transfer. It is supported if the AHB5 Secure_Transfers property is True.

3 Signal Descriptions

2. Slave signals

Name	Destination	Description
HRDATA[31:0]	Multiplexor	During read operations, the read data bus transfers data from the selected slave to the multiplexor. The multiplexor then transfers the data to the master. A minimum data bus width of 32 bits is recommended. However, this can be extended to enable higher bandwidth operation.
HREADYOUT	Multiplexor	When HIGH, the **HREADYOUT** signal indicates that a transfer has finished on the bus. It can be driven LOW to extend a transfer.
HRESP	Multiplexor	The transfer response, after passing through the multiplexor, provides the master with additional information on the status of a transfer. When LOW, the **HRESP** signal indicates that the transfer status is OKAY. When HIGH, the **HRESP** signal indicates that the transfer status is ERROR.
HEXOKAY	Multiplexor	Exclusive Okay. Indicates the success or failure of an Exclusive Transfer. It is supported if the AHB5 Exclusive_Transfers property is True.

3 Signal Descriptions

3. Decoder signals

Name	Destination	Description
HSELxa	Slave	Each slave has its own slave select signal **HSELx** and this signal indicates that the current transfer is intended for the selected slave. When the slave is initially selected, it must also monitor the status of **HREADY** to ensure that the previous bus transfer has completed, before it responds to the current transfer. The **HSELx** signal is a combinatorial decode of the address bus.

3 Signal Descriptions

4. Multiplexor signals

Name	Destination	Description
HRDATA[31:0]	Master	Read data bus, selected by the decoder.
HREADY	Master and slave	When HIGH, the **HREADY** signal indicates to the master and all slaves, that the previous transfer is complete.
HRESP	Master	Transfer response, selected by the decoder.
HEXOKAY	Master	Exclusive okay, selected by the decoder.

Agenda

AMBA AHB protocol

1 Introduction

2 About the protocol

3 Signal Descriptions

4 Transfers

5 Bus Interconnection

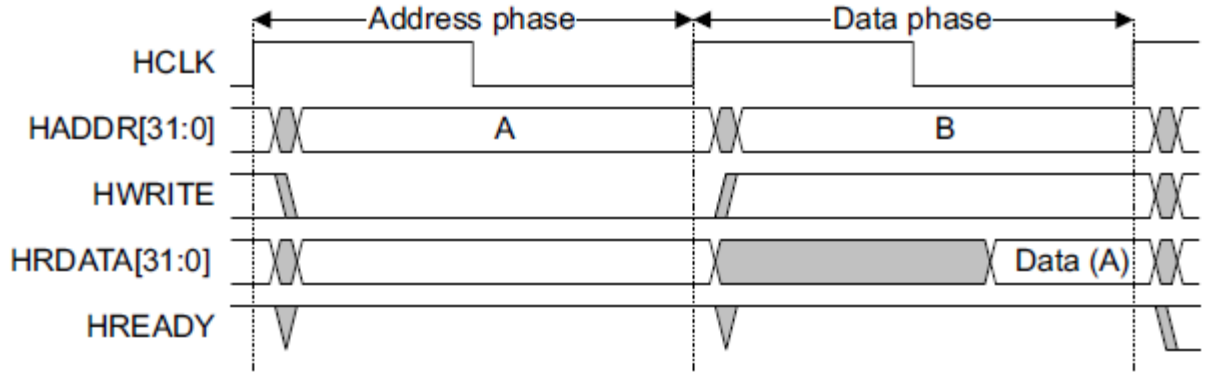
6 Slave Response Signaling

4 Transfers

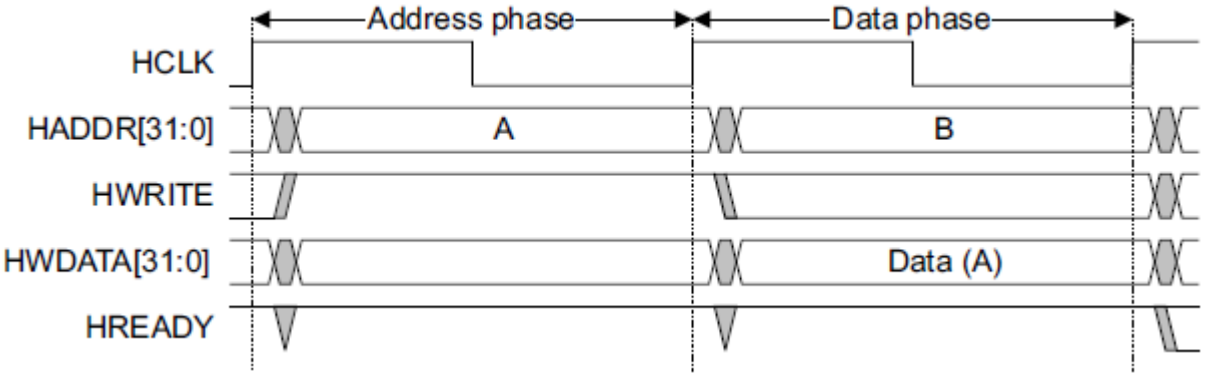
4.1 Basic Transfers

Simple transfer with no wait states:

- A transfer consists of two phases:
  - Address: Require a single HCLK cycle unless it's extended by the previous transfer.
  - Data: Might require several HCLK cycles. Use the **HREADY** signal to control the number of clock cycles required to complete the transfer.
- The simplest transfer is one with no wait states, so the transfer consists of one address cycle and one data cycle.



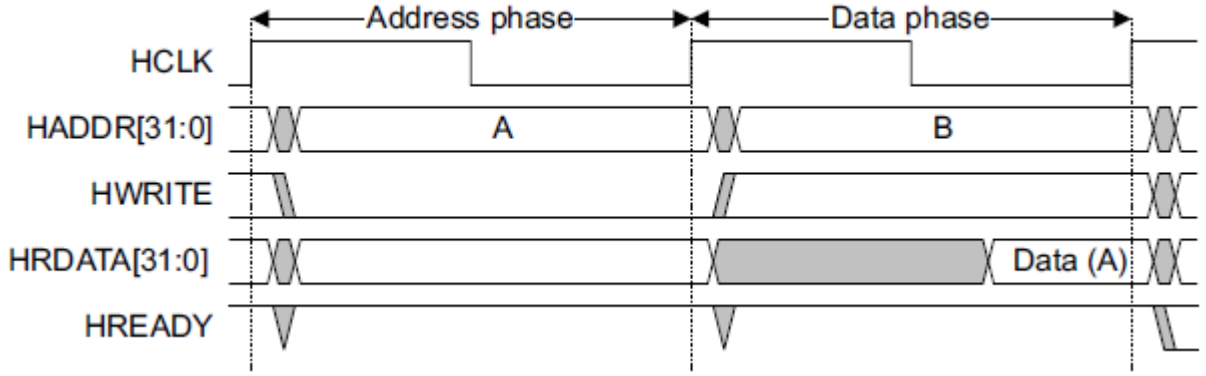
Read Transfer



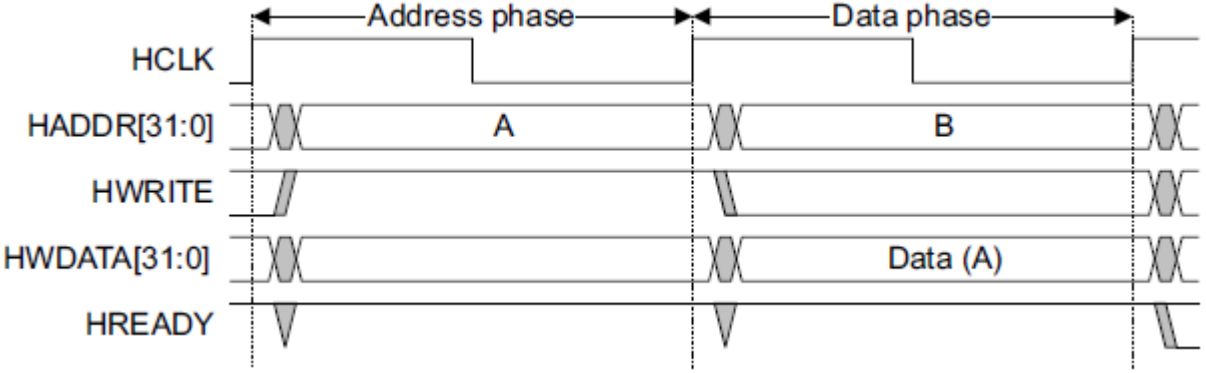
Write Transfer

4.1 Basic Transfers

- In a simple transfer with no wait states:
  1. Master drives the address and control signals after the rising edge of HCLK.
  2. Slave samples the address and control information on the next rising edge of HCLK.
  3. Slave drive the HREADYOUT response, which is sampled by the master on the third rising edge of HCLK.



Read Transfer



Write Transfer

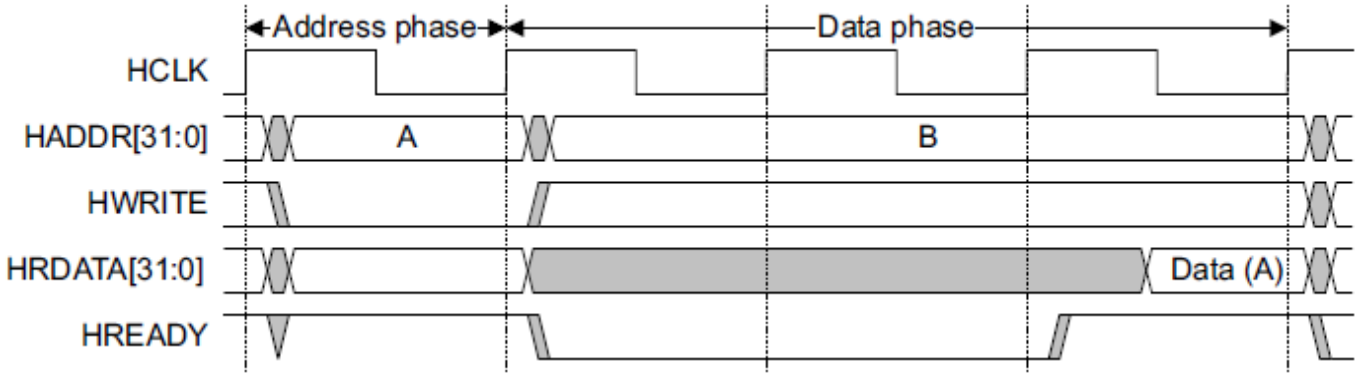
4.1 Basic Transfers

- This simple example demonstrates how the address and data phases of the transfer occur during different clock cycles.
- The AHB bus enables the overlapping of address and data phases (**pipeline feature**) as the address phase of any transfer occurs during the data phase of the previous transfer.
- This pipelined nature of AHB bus enables high performance operation while still providing adequate time for a slave to provide the response to a transfer.
- A slave can insert **wait states** into any transfer to enable additional time for completion.
- Each slave has an HREADYOUT signal that it drives during the data phase of a transfer.
- The interconnect is responsible for combining the HREADYOUT signals from all slaves to generate a single HREADY signal that is used to control the overall progress.

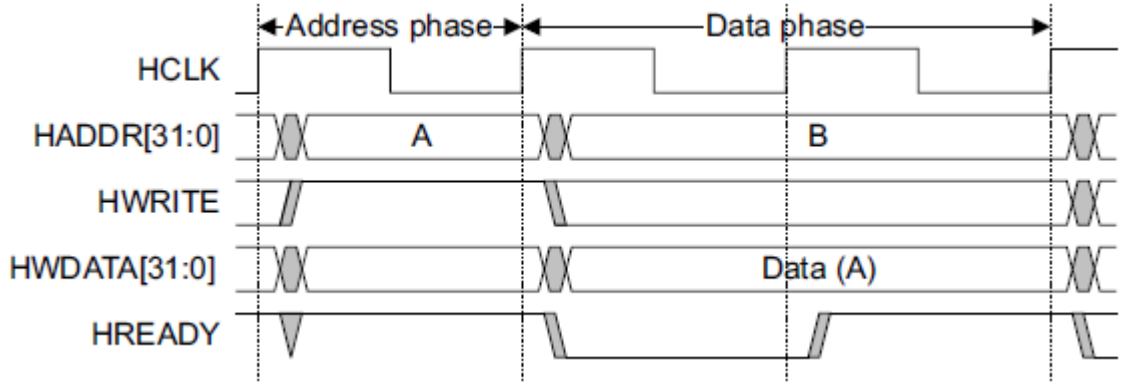
4.1 Basic Transfers

Simple transfer with wait states:

- For write operations the master holds the data stable throughout the extended cycles.
- For read transfers the slave does not have to provide valid data until the transfer is about to complete.
- When a transfer is extended, the address phase of the next transfer will be extended.



a read transfer with two wait states.

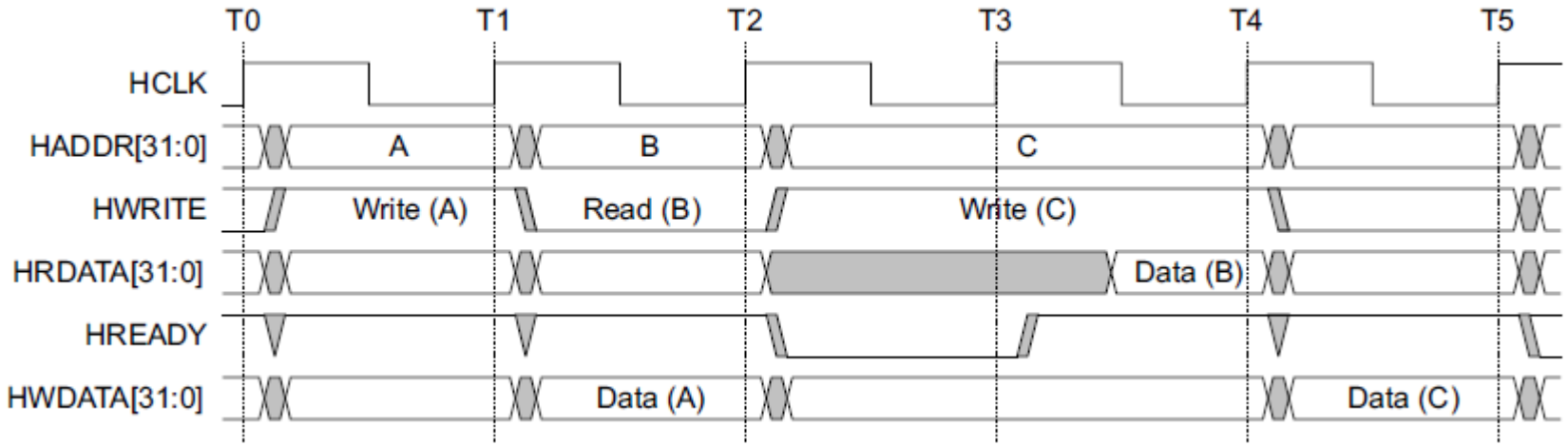


a write transfer with one wait state.

4.1 Basic Transfers

Multiple transfers:

- The transfers to addresses A and C are zero wait state
- The transfer to address B is one wait state
- Extending the data phase of the transfer to address B has the effect of extending the address phase of the transfer to address C.



Multiple transfers

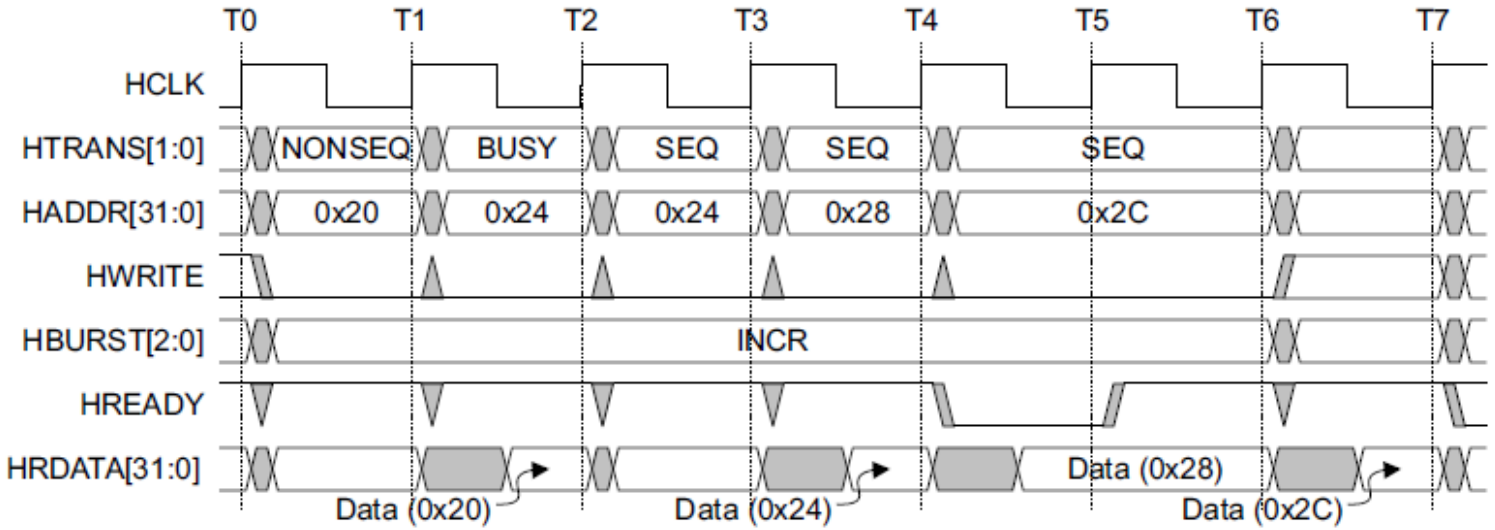
4.2 Transfer types

HTRANS[1:0]	Type	Description
0b00	IDLE	- Indicates that no data transfer is required. - Slaves must always provide a zero-wait state OKAY response to IDLE transfers and the transfer must be ignored by the slave.
0b01	BUSY	- It enables masters to insert idle cycles in the middle of a burst. - This transfer type indicates that the master is continuing with a burst, but the next transfer cannot take place immediately. - When a master uses the BUSY transfer type, the address and control signals must reflect the next transfer in the burst. - Only undefined length bursts can have a BUSY transfer as the last cycle. - Slaves must always provide a zero-wait state OKAY response to BUSY transfers and the transfer must be ignored by the slave.

4.2 Transfer types

HTRANS[1:0]	Type	Description
0b10	NONSEQ	- Indicates a single transfer or the first transfer of a burst. - The address and control signals are unrelated to the previous transfer. - Single transfers on the bus are treated as bursts of length one and therefore the transfer type is NONSEQUENTIAL.
0b11	SEQ	- The remaining transfers in a burst are SEQUENTIAL and the address is related to the previous transfer. - The control information is identical to the previous transfer. - The address is equal to the previous transfer address plus the transfer size, in bytes. - In the case of a wrapping burst the address of the transfer wraps at the address boundary.

4.2 Transfer types



T0-T1	The 4-beat read starts with a NONSEQ transfer.
T1-T2	The master is unable to perform the second beat and inserts a BUSY transfer to delay the start of the second beat. The slave provides the read data for the first beat.
T2-T3	The master is now ready to start the second beat, so a SEQ transfer is signaled. The master ignores any data that the slave provides on the read data bus.
T3-T4	The master performs the third beat. The slave provides the read data for the second beat.
T4-T5	The master performs the last beat. The slave is unable to complete the transfer and uses **HREADYOUT** to insert a single wait state.
T5-T6	The slave provides the read data for the third beat.
T6-T7	The slave provides the read data for the last beat.

4.4 Transfer size

- **HSIZE[2:0]** indicates the size of a data transfer.
- The transfer size set by **HSIZE** must be less than or equal to the width of the data bus.
  For example, with a 32-bit data bus, **HSIZE** must only use the values 000, 001, or 010.
- Using **HSIZE** with **HBURST**, to determine the address boundary for wrapping bursts.
- The **HSIZE** signals have exactly the same timing as the address bus. However, they must remain constant throughout a burst transfer.

HSIZE[2:0]	Size (bits)	Description
000	8	Byte
001	16	Halfword
010	32	Word
011	64	Doubleword
100	128	4-word line
101	256	8-word line
110	512	-
111	1024	-

4.5 Burst operation

- AHB supports the **incrementing bursts** by accessing sequential locations and the address of each transfer in the burst is an increment of the previous address.
- AHB supports the **wrapping bursts** by wrapping when they cross an address boundary.
- The address boundary is calculated as the product of the number of beats in a burst (**HBURST**) and the transfer size (**HSIZE**).
- For example, a four-beat wrapping burst of word (4-byte) accesses wraps at 16-byte boundaries. Therefore, if the start address of the burst is 0x34, then it consists of four transfers to addresses 0x34, 0x38, 0x3C, and 0x30.

HBURST[2:0]	Type	Description
000	SINGLE	Single transfer burst
001	INCR	Incr burst of undefined length
010	WRAP4	4-beat wrapping burst
011	INCR4	4-beat incrementing burst
100	WRAP8	8-beat wrapping burst
101	INCR8	8-beat incrementing burst
110	WRAP16	16-beat wrapping burst
111	INCR16	16-beat incrementing burst

4.5 Burst operation

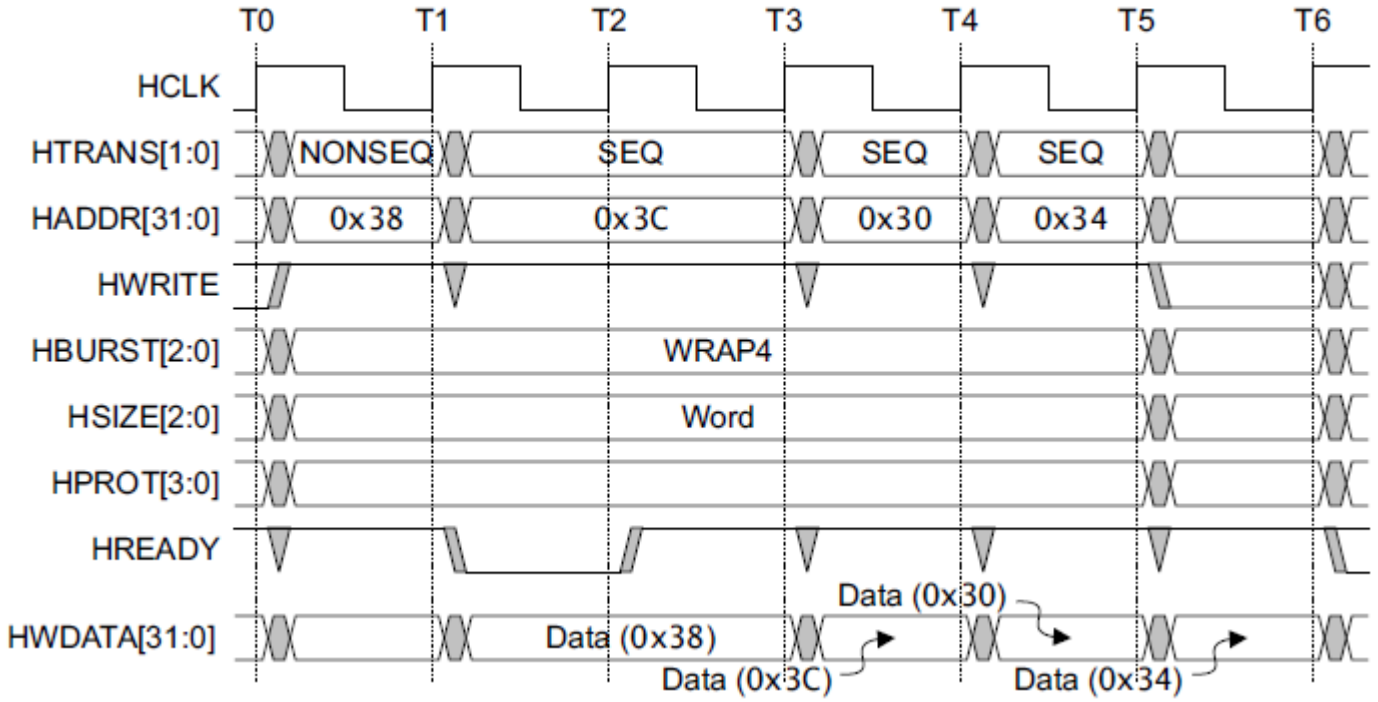
- Masters must not attempt to start an incrementing burst that crosses a 1KB address boundary.
- Masters can perform single transfers using either:
  - SINGLE transfer burst.
  - Undefined length burst that has a burst of length one.

- The burst size indicates the number of beats in the burst and not the number of bytes transferred.
  Calculate the total amount of data transferred in a burst by multiplying the number of beats by the amount of data in each beat, as indicated by **HSIZE[2:0]**.
- All transfers in a burst must be aligned to the address boundary equal to the size of the transfer.
  For example, word transfers must align to word address boundaries (**HADDR[1:0]** = 0b00), and halfword transfers to halfword address boundaries (**HADDR[0]** = 0).

4.5 Burst operation

Four-beat wrapping burst, WRAP4

- A write transfer using a four-beat wrapping burst, with a wait state added for the first transfer.
- Because the burst is a four-beat burst of word transfers, the address wraps at 16-byte boundaries, and the transfer to address 0x3C is followed by a transfer to address 0x30.



Wrapping Window:

0011_1100 0x3C

0011_1000 => Start (0x38)

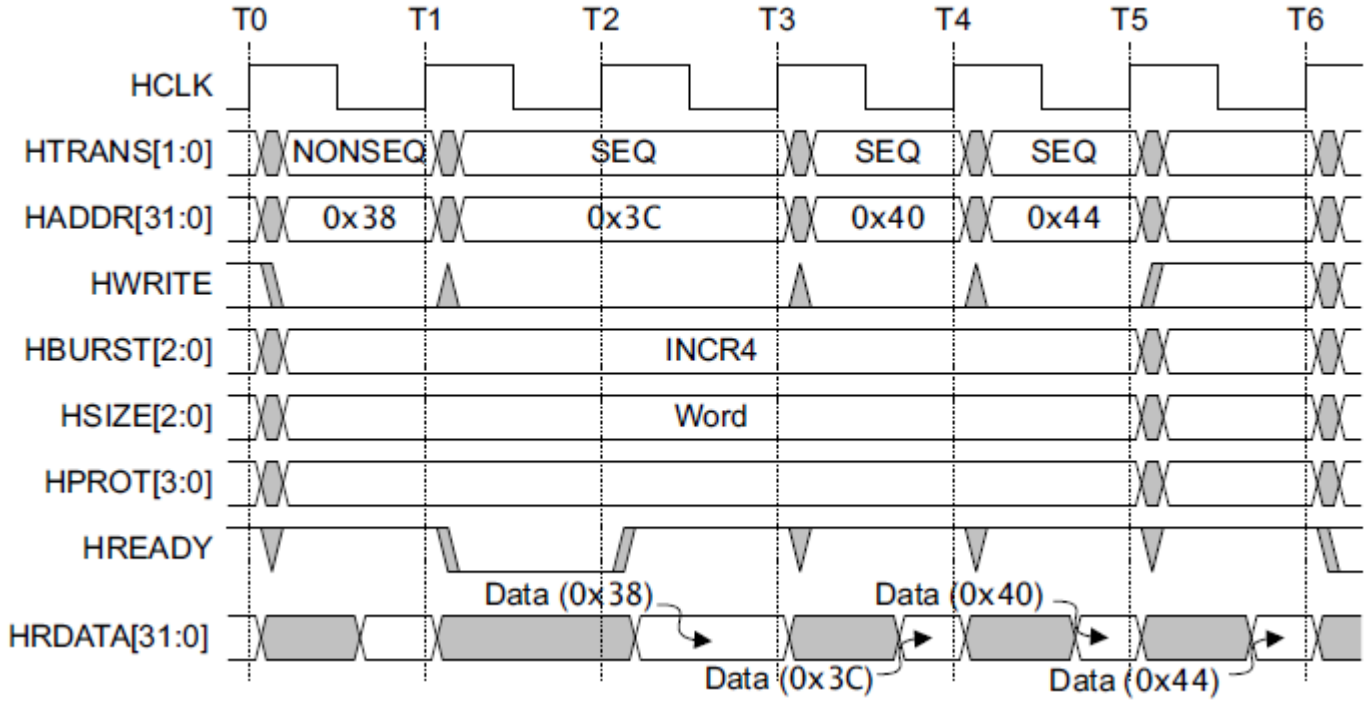
0011_0100 0X34

0011_0000 0X30

4.5 Burst operation

Four-beat incrementing burst, INCR4

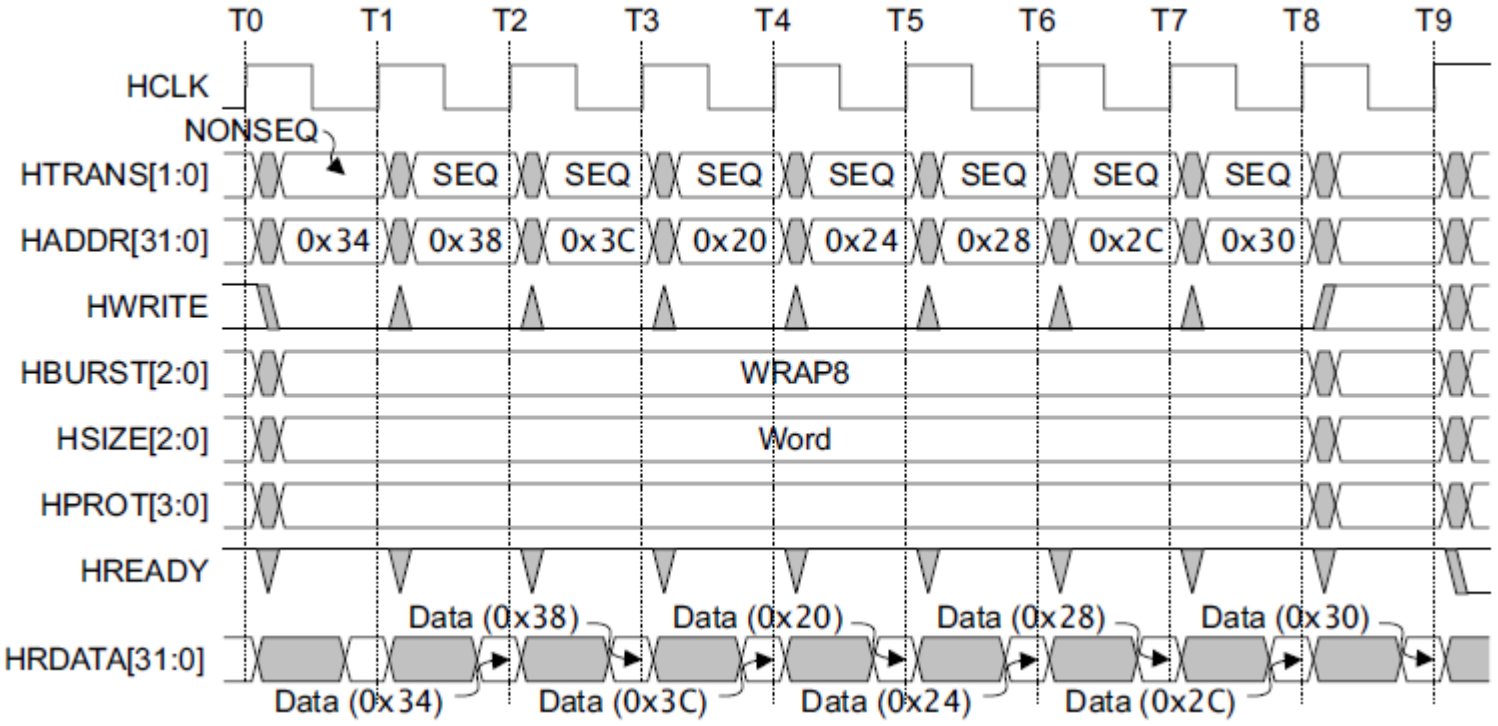
- A read transfer using a four-beat incrementing burst, with a wait state added for the first transfer.
- In this case, the address does not wrap at a 16-byte boundary and the address 0x3C is followed by a transfer to address 0x40.



4.5 Burst operation

Eight-beat wrapping burst, WRAP8

- A read transfer using an eight-beat wrapping burst.
- Because the burst is an eight-beat burst of word transfers, the address wraps at 32-byte boundaries, and the transfer to address 0x3C is followed by a transfer to address 0x20.



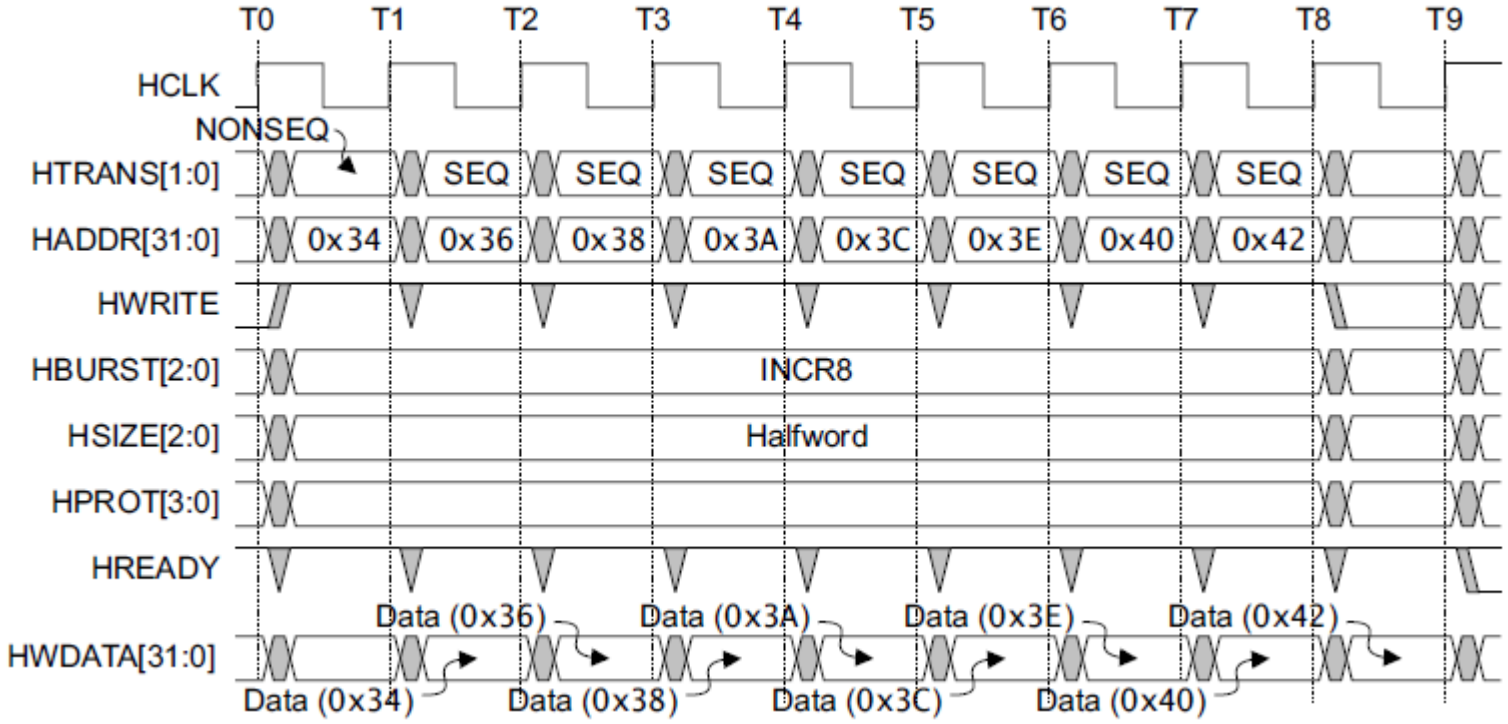
Wrapping Window:

0011_1100	0x3C
0011_1000	0x38
0011_0100	=> Start (0x34)
0011_0000	0X30
0010_1100	0X2C
0010_1000	0X28
0010_0100	0X24
0010_0000	0X20

4.5 Burst operation

Eight-beat incrementing burst, INCR8

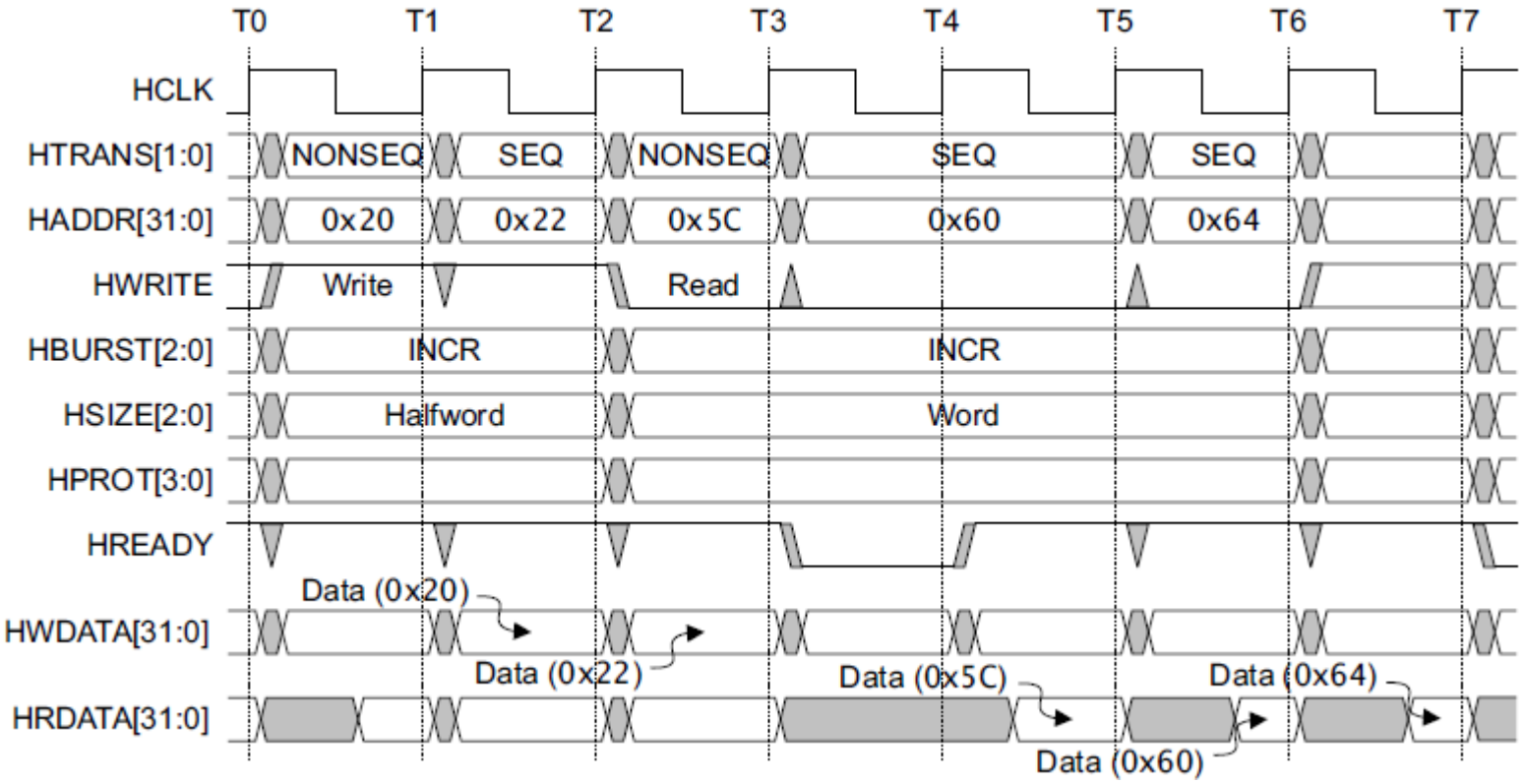
- A write transfer using an eight-beat incrementing burst.
- This burst uses halfword transfers, therefore the addresses increase by two. Because the burst is incrementing, the addresses continue to increment beyond the 16-byte address boundary.



4.5 Burst operation

Undefined length bursts, INCR

- The first burst is a write consisting of two halfword transfers starting at address 0x20. Its addresses increment by two.
- The second burst is a read consisting of three word transfers starting at address 0x5C. Its addresses increment by four.



Agenda

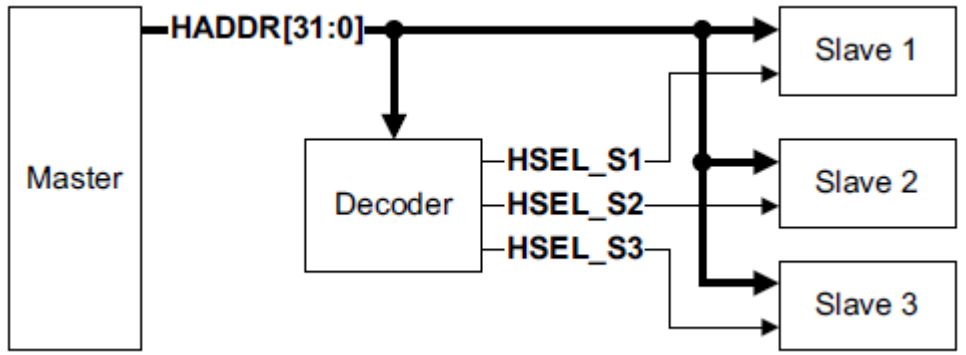
AMBA AHB protocol

- # 1 Introduction
- # 2 About the protocol
- # 3 Signal Descriptions
- # 4 Transfers
- # 5 Bus Interconnection
- # 6 Slave Response Signaling

5 Bus Interconnection

5.2 Address decoding

- An address decoder provides a **HSELx** for each slave on the bus.
- The select signal is a combinatorial decode of the address signals.
- A slave must only sample the **HSELx**, address, and control signals when **HREADY** is HIGH, indicating that the current transfer is completing.
- Under certain circumstances it is possible that **HSELx** is asserted when **HREADY** is LOW, but the selected slave has changed by the time the current transfer completes.



- The minimum address space that can be allocated to a single slave is 1KB, and the start and the end of the address region must exist on a 1KB boundary.
- All masters are designed so that they do not perform incrementing transfers over a 1KB address boundary. This ensures that a burst never crosses an address decode boundary.

5.2 Address decoding

1. Default slave

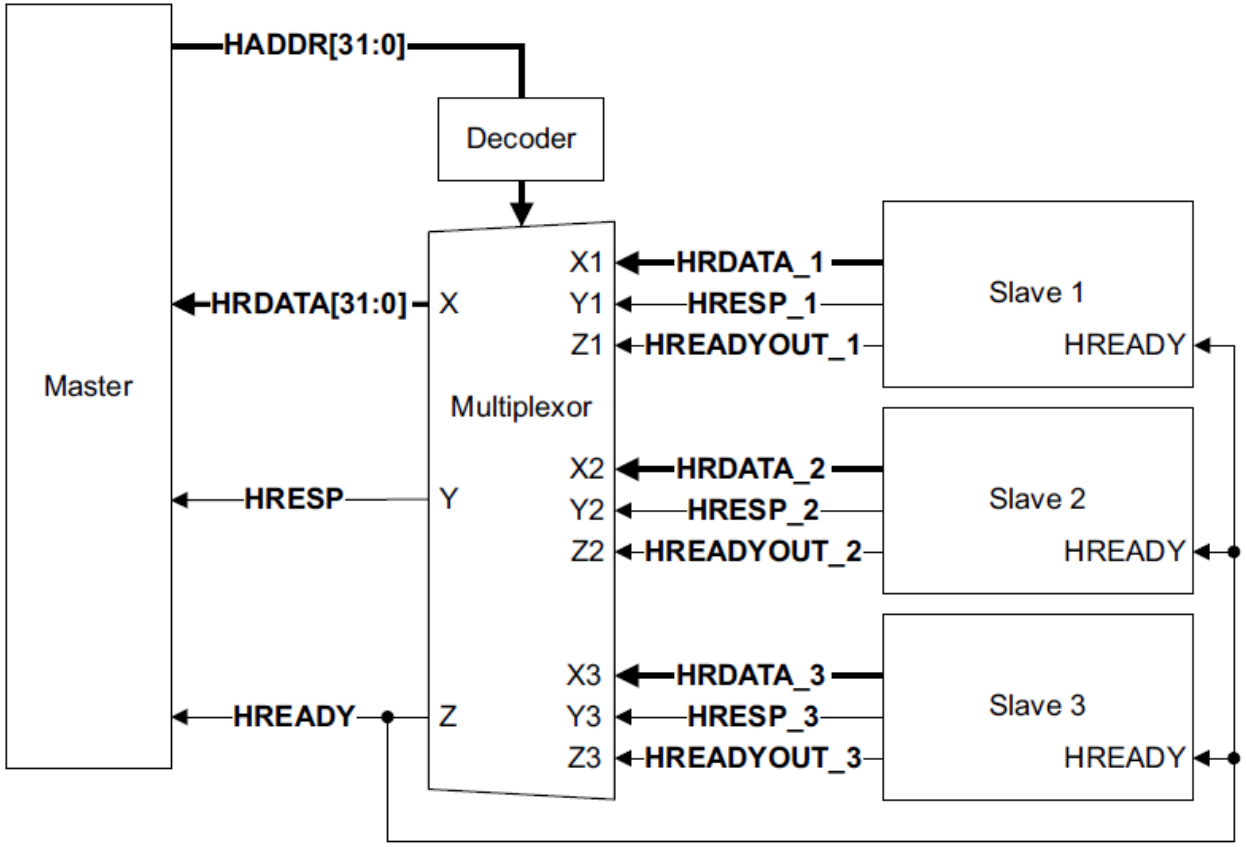
- If a system design does not contain a completely filled memory map, then an additional default slave must be implemented to provide a response when any of the nonexistent address locations are accessed.
- If a NONSEQUENTIAL or SEQUENTIAL transfer is attempted to a nonexistent address location, then the default slave provides an ERROR response.
- IDLE or BUSY transfers to nonexistent locations result in a zero-wait state OKAY response.

2. Multiple slave select

- A single slave interface is permitted to support multiple slave select, **HSELx**, signals. Each **HSELx** signal corresponds to a different decode of the higher order address bits.
- This permits a single slave interface to provide multiple logical interfaces, each with a different location in the system address map. The minimum address space that can be allocated to a logical interface is 1KB.
- It can be used in a peripheral that has its main data path and control registers at different locations in the address map. Both locations can be accessed through a single interface without performing an address decode.

5.3 Read data and response multiplexor

- The AHB protocol is used with a read data multiplexor interconnection scheme.
- The master drives out the address and control signals to all slaves, with the decoder selecting the appropriate slave.
- Any response data from the selected slave, passes through the read data multiplexor to the master.



Agenda

AMBA AHB protocol

1 Introduction

2 About the protocol

3 Signal Descriptions

4 Transfers

5 Bus Interconnection

6 Slave Response Signaling

6 Slave Response Signaling

5 Slave transfer responses

- After a master has started a transfer, the slave controls how the transfer progresses. A master cannot cancel a transfer after it has commenced.
- A slave must provide a response that indicates the status of the transfer when it is accessed. The transfer status is provided by the **HRESP** signal.

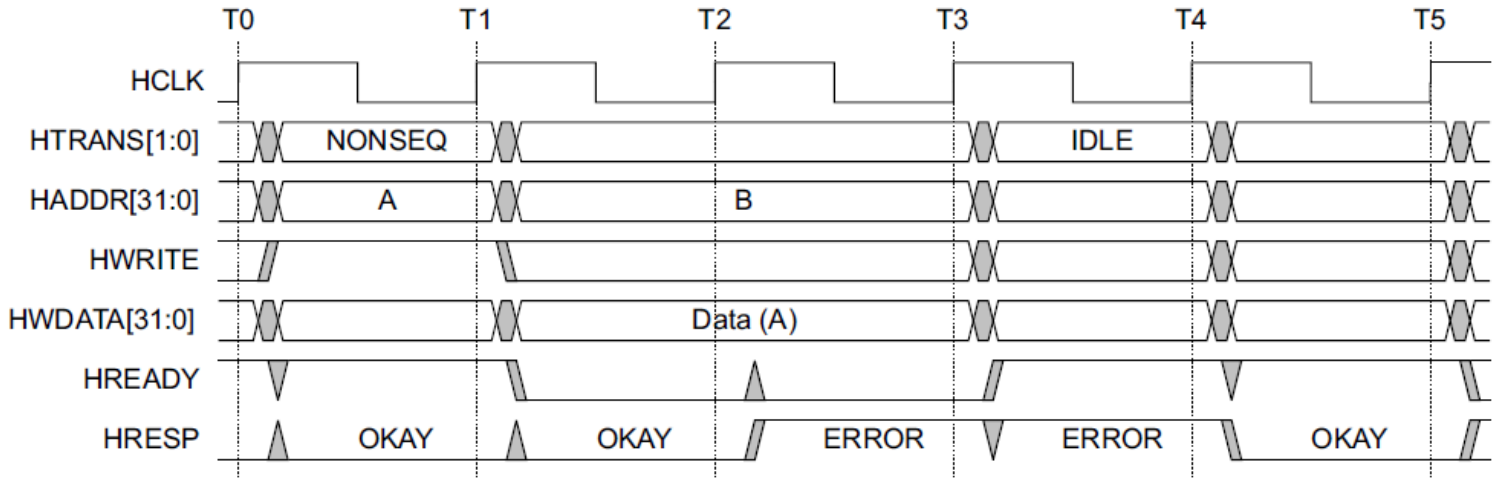
HRESP	Response	Description
0	OKAY	The transfer has either completed successfully or additional cycles are required for the slave to complete the request. The **HREADYOUT** signal indicates whether the transfer is pending or complete.
1	ERROR	An error has occurred during the transfer. The error condition must be signaled to the master so that it is aware the transfer has been unsuccessful. A two-cycle response is required for an error condition with **HREADYOUT** being asserted in the second cycle.

5 Slave transfer responses

- This means the slave can complete the transfer in the following three ways:
  - Immediately complete the transfer.
  - Signal an error to indicate that the transfer has failed.
  - Insert one or more wait states to enable time to complete the transfer.

	HREADYOUT = 0	HREADYOUT = 1
HRESP = 0	Transfer pending	Successful transfer completed
HRESP =1	ERROR response, first cycle	ERROR response, second cycle

5 Slave transfer responses



T1-T2	The slave inserts a wait state and provides an OKAY response.
T2-T3	The slave issues an ERROR response. This is the first cycle of the ERROR response because **HREADY** is LOW.
T3-T4	The slave issues an ERROR response. This is the last cycle of the ERROR response because **HREADY** is now HIGH. The master changes the transfer type to IDLE. This cancels the intended transaction to address B, that was registered by a slave at time T2.
T4-T5	Slave responds with an OKAY response.

If a slave provides an ERROR response, then the master can cancel the remaining transfers in the burst. However, this is not a strict requirement, and it is acceptable for the master to continue the remaining transfers in the burst.

This presentation is prepared from:

Arm® Cortex®-M System Design Kit
Revision: r1p1
Technical Reference Manual.

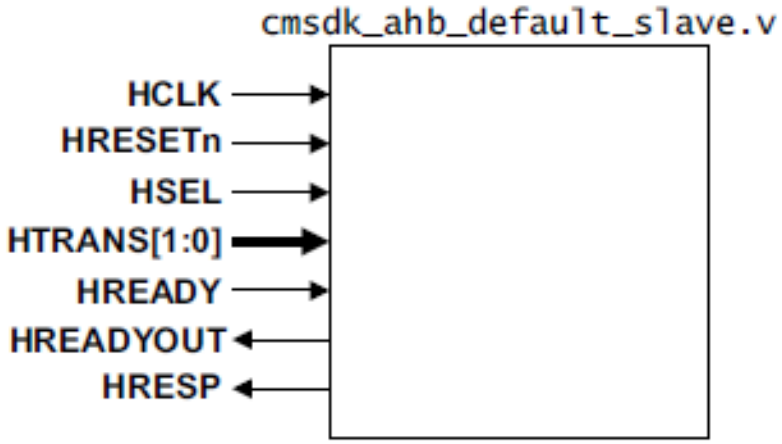
Agenda

Cortex®-M System Design Kit

- # 3 Basic AHB-Lite components
  - # 3.1 AHB default slave
  - # 3.5 AHB GPIO

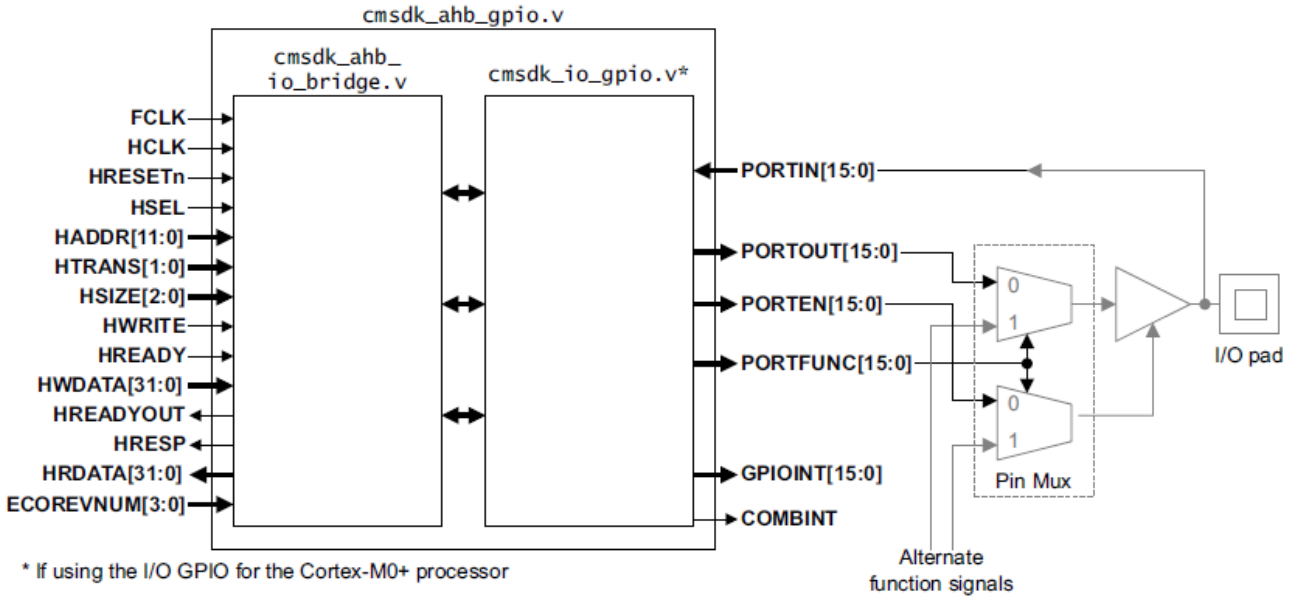
3.1 AHB default slave

- The AHB default slave, cmsdk_ahb_default_slave.v, responds to transfers when the bus master accesses an undefined address.
- A zero-wait state OKAY response is generated for IDLE or BUSY transfers, and an ERROR response is generated for NONSEQUENTIAL or SEQUENTIAL transfers.



3.5 AHB GPIO

- The AHB GPIO, cmsdk_ahb_gpio.v, is a general-purpose I/O interface unit.
- The AHB GPIO provides a 16-bit I/O interface with the following properties:
  - Programmable interrupt generation capability.
  - Bit masking support using address values.
  - Registers for alternate function switching with pin multiplexing.
  - Thread safe operation by providing separate set and clear addresses for control registers.
  - Inputs are sampled using a double flip-flop to avoid metastability issues.



3.5 AHB GPIO

3.5.2 Programmers model

Name	Base offset	Type	Width	Reset	value Description
DATA	0x0000	RW	16	0x----	Data value [15:0]: **Read** Sampled at pin. **Write** To data output register. Read back value goes through double flip-flop synchronization logic with a delay of two cycles.
DATAOUT	0x0004	RW	16	0x0000	Data output Register value [15:0]: **Read** Current value of data output register. **Write** To data output register.
OUTENSET	0x0010	RW	16	0x0000	Write 1 to set the output enable **0** Indicates the signal direction as input. **1** Indicates the signal direction as output.
OUTENCLR	0x0014	RW	16	0x0000	Write 1 to clear the output enable **Read back 0** Indicates the signal direction as input, otherwise it's an output.
ALTFUNCSET	0x0018	RW	16	0x0000	Write 1 to set the Alternative function **Read back 0** For I/O, otherwise for an alternate function.
ALTFUNCCLR	0x001C	RW	16	0x0000	Write 1 to clear the Alternative function **Read back 0** For I/O, otherwise for an alternate function.

3.5 AHB GPIO

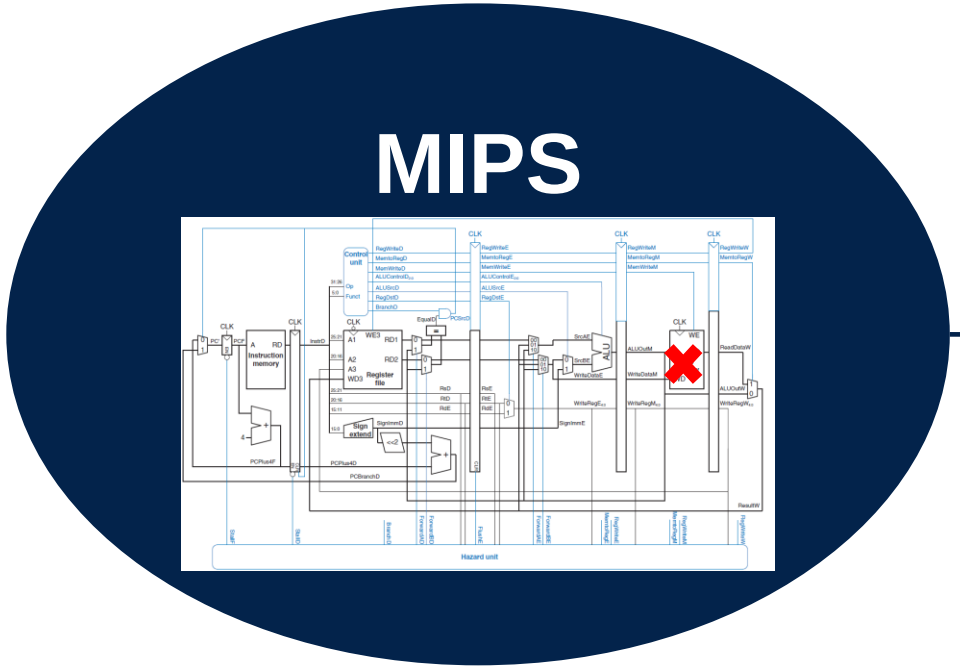
- **3.5.2 Programmers model**

Name	Base offset	Type	Width	Reset	value Description
INTENSET	0x0020	RW	16	0x0000	Interrupt enable set [15:0]: **Write 1** to set the enable bit. **Read back 0** Interrupt disabled, or **1** Interrupt enabled.
INTENCLR	0x0024	RW	16	0x0000	Interrupt enable clear [15:0]: **Write 1** to clear the enable bit. **Read back 0** Interrupt disabled, or **1** Interrupt enabled.
INTTYPESET	0x0028	RW	16	0x0000	Interrupt type set [15:0]: **Write 1** Sets the interrupt type bit. **Read back 0** For LOW/HIGH level, or **1** For falling edge/rising edge.
INTTYPECLR	0x002C	RW	16	0x0000	Interrupt type clear [15:0]: **Write 1** Clears the interrupt type bit. **Read back 0** For LOW/HIGH level, or **1** For falling/rising edge.

Assignment #7

Assignment #7

- Our target is to implement the shown architecture.



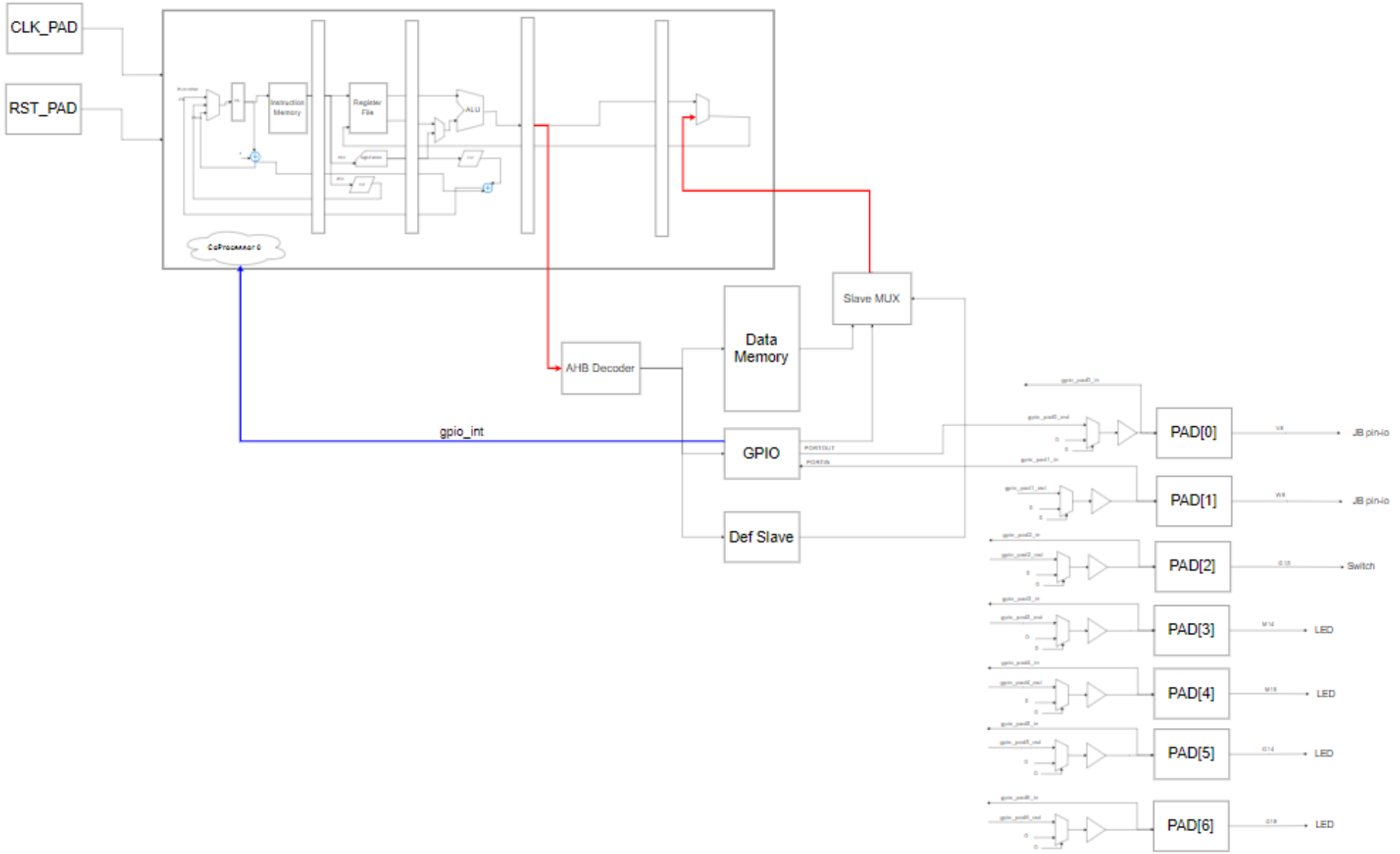
AHB Decoder

Data Memory

GPIO

Def slave

Assignment #7



Assignment #7

- Tie MIPS with AHB port and connect it to data memory and GPIO via a decoder.
- Do a presentation to explain your work (15 mins).
- Deadline: Friday, 13th Dec 2024, at 11:59 PM.
- Deliveries: Full design, testbench, a document and a presentation for your work.

Assignment #7

Our Peripheral Addresses start from 0xA000_0000 (Virtual Address), so, Divide the top Reserved Area into:

- From 0x8000_0000 till 0x9FFF_FFF: (0x8000 till 0x9FFF, as physical addresses) will be reserved for OS, Exceptions Handler starts from 0x0x8000_0180 (0x0x8180, as physical address) [Add this portion as instruction memory]
- From 0xA000_0000 till 0XFFFF_FFFF: will be reserved for the peripheral: [This portion should be occurred inside each peripheral as registers, not all the range for sure].
  - 0xA000_0000 till 0xA000_03FF: For GPIO
  - 0xA000_0400 till 0xA000_07FF: For UART
  - 0xA000_0800 till 0xA000_0BFF: For SPI
  - 0xA000_0C00 till 0xA000_0FFF: For TIMER
  - 0xA000_1000 till 0xFFFF_FFFF: For def_slave
- No need to translate the addresses of the peripheral, we just subtract its base address.

Assignment #7

Test:

- lui $s0, 0xA000
- ori $s0, $s0, 0x0000
- lui $s1, 0x0000
- ori $s1, $s1, 0x5A5A
- sw $s1, 0x0000($s0)
- ori $s1, $s1, 0xFFFF
- sw $s1, 0x0010($s0)

We’re writing 0x0000_5A5A in GPIO peripheral at DATA register whose offset is 0x0000 and configuring them as output by setting the OUTENSET register whose offset is 0x0010.